

# EdgeInfer: Hardware-Aware Efficient Execution Planning for High-Performance LLM Inference on Static-Graph Edge NPUs

## Abstract

Dynamic sequence lengths make LLM inference a poor fit for edge neural processing units (NPUs) whose efficiency relies on ahead-of-time static graphs. Fixed-Max deployment pads every request to the maximum context, wasting computation. Real-device measurements of static Attention on two mobile NPUs reveal three obstacles: useful causal geometry changes with inference state; the fastest graph granularity is device-dependent and is not identified reliably by FLOPs, launch count, or padding; and independently executed blocks remain coupled by global softmax normalization. We present EdgeInfer, a hardware-aware execution planner that addresses these obstacles with reusable static Attention blocks, real-device cost-guided planning, and exact state composition. Its perceive–decide–execute loop measures a finite block library, caches the lowest-cost legal plan for each inference state, and executes only the selected immutable graphs through Qualcomm AI Engine Direct. Over the PyTorch ExecuTorch Fixed-Max baseline, EdgeInfer accelerates one 40K decoder block on Xiaomi 13 SM8550 by  $76.9\times$  in Decode and  $39.2\times$  in Prefill; on OnePlus 12 SM8650, the corresponding speedups are  $93.4\times$  and  $46.5\times$ .

## 1 Introduction

On-device LLM inference is practical, but efficient use of mobile accelerators now limits deployment. Local execution preserves privacy and avoids cloud latency, while EdgeLLM reduces generation cost (Xu et al. 2025a). Compact architectures enable phone-scale models (Liu et al. 2024b), and mobile quantization and low-bit kernels reduce their resource demand (Tan et al. 2024; Wei et al. 2025). The question is no longer whether an LLM can run on a phone, but whether its evolving workload can use the phone’s most energy-efficient tensor engine.

The central obstacle is a mismatch between dynamic Attention geometry and static NPU execution. Mobile NPUs obtain high efficiency from ahead-of-time compiled graphs. llm.npu improves their utilization by reorganizing prompts, tensors, and Transformer blocks (Xu et al. 2025b), while heterogeneous systems map inference phases across CPUs, GPUs, and NPUs (Chen et al. 2025). These systems demonstrate the potential of mobile NPUs, yet prompt lengths still vary across requests and the visible key/value (K/V) context still grows after every generated token. A graph compiled

Code, supplementary materials, and reproducibility resources are available at <https://github.com/pngbit/executorch-edgeinfer-npu-preview>.

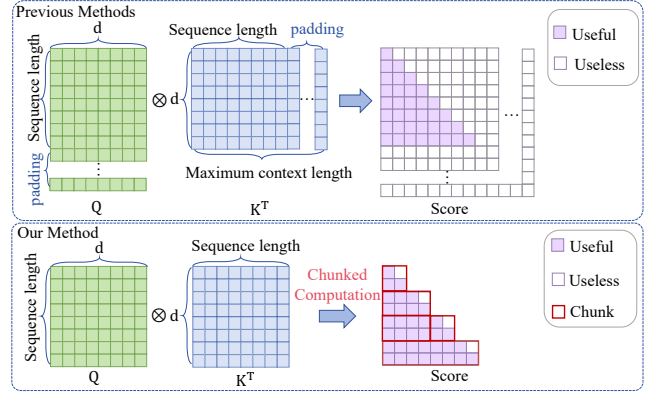


Figure 1: Fixed-Max pads Attention to 40K; EdgeInfer executes only static blocks covering the valid causal region (red outlines).

for the maximum context therefore cannot contract with the useful workload.

Fixed-Max deployment makes this mismatch concrete. Figure 1 shows that padding and causal masks preserve numerical semantics but cannot remove matrix operations already compiled into  $QK^T$  and the subsequent value projection. A fixed 40K graph consequently computes padded Query rows, unused K/V columns, and the causally invalid upper-right region even when only a small fraction is useful. Mobile-NPU operators reduce phase-specific cost (Xu et al. 2024), and heterogeneous scheduling improves hardware mapping (Chen et al. 2025); neither provides a composable static execution unit that follows changing Attention geometry.

**Research question (RQ).** How can a static-graph edge NPU adapt to evolving LLM Attention workloads, without dynamic-shape hardware support, while preserving exact Attention semantics and low real-device latency?

Real-device measurements reveal three reasons why existing static deployment falls short. **(1) Dynamic-geometry mismatch:** a fixed maximum-context graph performs substantial padded and causally invalid computation. **(2) Device-dependent granularity:** latency is non-monotonic in block area; neither fewer floating-point operations, fewer graph launches, nor less padding consistently identifies the fastest

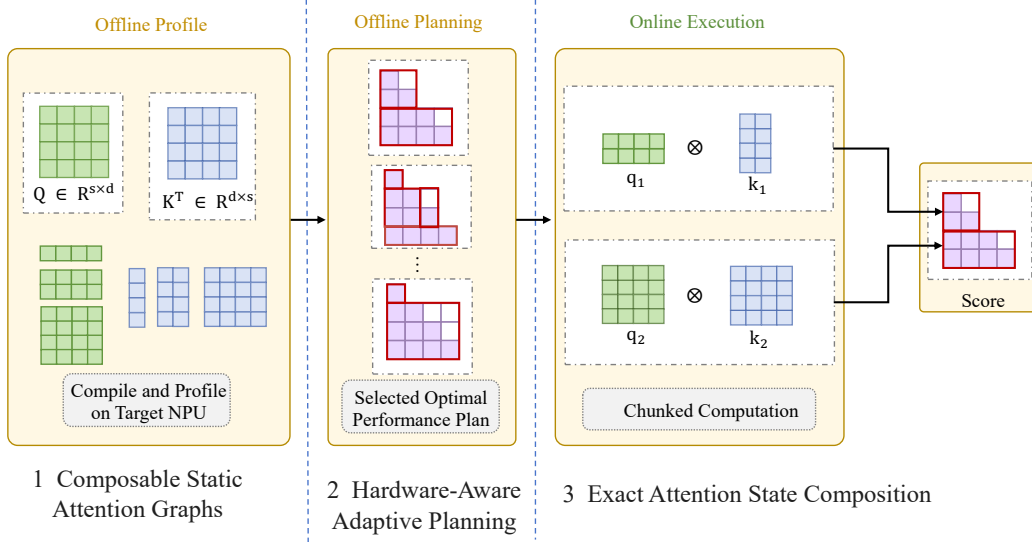


Figure 2: EdgeInfer’s perceive–decide–execute loop. (1) Composable Static Attention Graphs compile reusable  $G(R, C)$  blocks and measure their NPU costs. (2) Hardware-Aware Adaptive Planning caches  $P^*(x)$  for state  $x = (\text{phase}, R, S)$ , where  $S$  is visible K/V length. (3) Exact Attention State Composition executes the selected blocks through  $QK^T$ , local softmax statistics, and multiplication by  $V$ , then merges their states into one global softmax. The dashed branch marks the backend-independent extension point for future GPU/CPU placement.

shape. (3) **Global normalization dependency:** K/V blocks may compute local scores independently, but independently normalized outputs cannot be concatenated because every Query row shares one softmax denominator. Together, these findings show that efficient execution requires both a composable representation and hardware-aware selection.

These findings map directly to EdgeInfer’s three-stage loop in Figure 2. **First, Composable Static Attention Graphs perceive hardware cost:** EdgeInfer compiles reusable  $G(R, C)$  blocks and measures each shape once on the target NPU. **Second, Hardware-Aware Adaptive Planning makes the execution decision:** it searches legal fused, padded, and multi-block candidates and caches the lowest-cost plan for every inference state. **Third, Exact Attention State Composition realizes that decision:** selected blocks execute as immutable NPU graphs, then their local maximum, exponential sum, and weighted numerator are merged into one global softmax. This perceive–decide–execute loop turns a dynamic LLM workload into a sequence of efficient static decisions without requiring dynamic-shape hardware.

Our contributions are summarized as follows:

- We identify and empirically characterize execution-granularity mismatch between dynamic LLM Attention and static-graph edge NPUs, showing why simple compute- or launch-count heuristics are insufficient.
- We introduce composable static Attention graphs and an associative state-composition rule that jointly express changing Query/K/V geometry while preserving exact global softmax semantics.
- We develop a hardware-aware adaptive planner and implement it with Qualcomm QNN. Evaluations on two real

mobile SoCs demonstrate substantial long-context Decode and Prefill speedups and stable scaling across stacks of Transformer decoder blocks.

## 2 Related Work

**On-device LLM inference.** The Transformer established Attention as the core sequence operator (Vaswani et al. 2017). Post-training quantization reduces model arithmetic and memory cost (Dettmers et al. 2022; Frantar et al. 2023; Xiao et al. 2023), while activation-aware and serving-oriented designs improve low-bit execution (Lin et al. 2024, 2025). LLM in a Flash addresses limited device memory (Alizadeh et al. 2024); ElasticLM adapts service capacity to mobile resources (Yin et al. 2025). Recent NPU-centric systems extend execution to background workloads and test-time scaling (Yin et al. 2026; Hao et al. 2026). These techniques complement the mobile systems discussed in Section 1, but do not supply a static NPU unit that follows request-dependent causal geometry.

**Attention and serving.** FlashAttention improves exact Attention locality and work partitioning (Dao et al. 2022; Dao 2024), while blockwise and ring execution distribute long contexts (Liu and Abbeel 2023; Liu, Zaharia, and Abbeel 2024). Attention sinks, heavy-hitter retention, and K/V quantization reduce active state (Xiao et al. 2024; Zhang et al. 2023; Liu et al. 2024a). Serving systems optimize cache management and iteration batching (Kwon et al. 2023; Yu et al. 2022; Sheng et al. 2023); phase-aware schedulers further improve throughput and goodput (Agrawal et al. 2024; Patel et al. 2024; Zhong et al. 2024). FlashInfer exposes customizable GPU kernels (Ye et al. 2025), whereas specu-

lative decoding and multiple heads reduce sequential generation steps (Leviathan, Kalman, and Matias 2023; Cai et al. 2024). These methods primarily target programmable server hardware; EdgeInfer instead composes ahead-of-time mobile-NPU graphs while preserving one exact softmax.

**Compiler and cost-guided planning.** TVM and Ansor automate target-aware tensor optimization (Chen et al. 2018; Zheng et al. 2020); TensorIR provides a tensorized scheduling abstraction (Feng et al. 2023). Nimble and Hidet address dynamic models and task mapping (Shen et al. 2021; Ding et al. 2023), while TASO and ROLLER optimize graph substitutions or construct accelerator-aligned programs (Jia et al. 2019; Zhu et al. 2022). EdgeInfer applies measurement-guided selection at a different abstraction: a context-specific composition of immutable Attention graphs that jointly preserves causal coverage, global normalization, and real-device latency.

### 3 Methodology

#### 3.1 Problem Formulation and Design Observations

A decoder-only LLM repeats Transformer decoder blocks, each containing pre-projection, Attention, and post-projection operations. The K/V cache of every block is independent, but all blocks experience the same evolving sequence geometry. We therefore optimize one Attention instance and reuse its static graph library and planning rule across the decoder stack. For an inference state  $x = (\phi, R, S)$ ,  $\phi \in \{\text{Decode}, \text{Prefill}\}$  is the phase,  $R$  is the number of Query rows in the current input, and  $S$  is the total visible K/V extent after those rows are appended. Thus Decode has  $R = 1$ , while a Prefill input has history length  $H_0 = S - R$ . Fixed-Max maps every  $x$  to one maximum-capacity graph  $C_{\max}$ ; EdgeInfer instead selects a legal composition  $P(x)$  whose executed geometry follows  $x$ .

**From an LLM to the planning unit.** Within each decoder block, the hidden state is projected to  $Q$ ,  $K$ , and  $V$ , Attention relates the current Query rows to the visible K/V cache, and an output projection and feed-forward network transform the result. The surrounding projections scale with the current Query block; Attention alone additionally couples those rows to the entire accumulated context. It is therefore the operator whose two-dimensional workload changes with both prompt progress and every Decode step. EdgeInfer places its adaptive boundary around this operator while leaving the decoder-block interface, per-block parameters, residual path, and independent K/V caches unchanged.

This formulation exposes three requirements. **Dynamic coverage** requires every visible Query-key pair, but no K/V position beyond the current context. **Exact normalization** requires all blocks serving one Query row to share a global softmax denominator. **Hardware efficiency** requires minimizing measured execution time rather than only FLOPs or graph count. These requirements lead to three observations and their corresponding mechanisms.

**Dynamic geometry requires a composable representation.** Padding preserves shape compatibility but does not

remove matrix operations compiled into  $QK^\top$  and the value projection. Decode changes its visible K/V width at every token, while Prefill additionally forms a lower-triangular causal region over multiple Query blocks. EdgeInfer therefore represents one request as a composition of immutable graphs rather than one padded maximum graph.

**Static shape cost must be perceived on hardware.** Changing Query rows or K/V width changes compiler tiling, vector utilization, graph submission, memory traffic, and state transfer simultaneously. Their combined latency is non-monotonic and differs across NPU generations. EdgeInfer consequently treats real-device measurements as the cost surface over which plans are selected, following the broader principle of measurement-guided tensor compilation (Chen et al. 2018; Zheng et al. 2020; Shen et al. 2021).

**Independent blocks require globally coupled execution.** K/V blocks can compute local scores in parallel, but independently normalized outputs cannot be concatenated. EdgeInfer retains a compact sufficient state for each block and merges these states exactly, building on the blockwise normalization principle used by exact Attention algorithms (Dao et al. 2022; Liu and Abbeel 2023).

Figure 2 summarizes the resulting perceive-decode-execute loop. Offline, EdgeInfer compiles and measures reusable blocks, then searches and caches the lowest-cost plan for every supported state. Online, it performs a constant-time plan lookup, launches only the selected static graphs, and merges their states. Hardware remains fully static; adaptation occurs in the software composition layer.

#### 3.2 Perception: Composable Static Attention Graphs

**Static block interface.** We denote a precompiled Attention graph by  $G(R, C)$ , where  $R$  is the Query-block height and  $C$  is the K/V-block width. It consumes  $Q_R$ ,  $K_C$ ,  $V_C$ , and the corresponding mask, and returns a local Attention state rather than a fully normalized output. The graph contains score computation, masking, local softmax statistics, and multiplication by  $V$ ; pre- and post-projections remain outside this interface. This boundary makes each graph reusable across requests, token positions, and Transformer decoder blocks.

The shape library is the Cartesian product of supported Query heights  $\mathcal{R}$  and K/V widths  $\mathcal{C}$ , after removing combinations rejected by the target compiler. Each surviving graph is compiled ahead of time and measured on the target NPU. The resulting cost table records the complete block path used by the planner, including graph execution and the state movement needed by that block. Unlike an analytic FLOP model, it preserves shape-specific compiler and hardware effects.

**Finite hardware cost surface.** Compilation turns every supported pair  $(R, C)$  into an immutable executable with a fixed tensor contract. Real-device measurement then attaches a latency to each executable role, including whether the block initializes a local state or merges with an existing one. The resulting table is a finite, device-specific cost surface shared by Decode and Prefill. It is measured once before deployment and remains separate from the request path, so online

adaptation does not require recompilation or dynamic-shape kernels.

**Geometry-aware composition.** For visible length  $S$ , Decode uses one Query row and a consecutive K/V chain whose widths cover  $S$ , whereas Prefill partitions both dimensions. Let Query block  $i$  cover offsets  $[a_i, b_i)$  within the current  $R$ -row input, and K/V block  $j$  cover global cache positions  $[u_j, v_j)$ . With  $H_0 = S - R$ , Prefill selects  $G(R_i, C_j)$  only when  $u_j < H_0 + b_i$ ; the Query heights partition  $[0, R)$  and the K/V widths cover  $[0, S)$ .

For Decode, only the final K/V block may extend beyond the visible cache, and its excess columns are masked; later blocks are not launched. For Prefill, Query block  $i$  is paired only with K/V blocks intersecting its visible prefix  $[0, H_0 + b_i)$ . A row-wise mask handles the partially visible diagonal block, while complete upper-right causal blocks are omitted. Thus every valid Query-key pair appears exactly once, and the physical workload closely follows the useful causal region in Figure 1.

This construction enforces three plan invariants. *Coverage* assigns every visible Query-key pair to a selected graph. *Exclusivity* assigns it exactly once, preventing duplicate probability mass. *Causality* excludes graph regions lying wholly beyond a Query block’s visible prefix and masks only unavoidable boundary cells. These invariants define plan legality independently of device speed; hardware measurements rank only plans that already satisfy them.

**Reuse across model depth.** Graph identity depends on  $(R, C)$  and the Attention contract, not on the decoder-block index. When model parameters are bound for each block, the same shape portfolio and context-indexed plan topology can be reused throughout the LLM. Each block retains its own K/V cache and state buffers, preserving the standard decoder computation while amortizing graph design and planning across model depth.

### 3.3 Decision: Hardware-Aware Adaptive Planning

**Measured plan space.** For state  $x$ , EdgeInfer constructs  $\mathcal{P}(x)$  from all legal fused graphs, exact-shape compositions, and lightly padded boundary compositions. A plan specifies Query partitioning, K/V widths, causal block coverage, execution order, and state-reduction topology. Keeping these alternatives in one space allows the planner to select the fastest legal path among all cases, including a single graph when it is optimal.

Let  $t_x(g)$  be the measured target-device cost of graph  $g$ , and let  $T_{\text{ovh}}(P)$  collect plan-level launch, memory, and state-merge costs. EdgeInfer chooses

$$P^*(x) = \arg \min_{P \in \mathcal{P}(x)} \left[ \sum_{g \in P} t_x(g) + T_{\text{ovh}}(P) \right]. \quad (1)$$

The planner realizes Equation 1 as a shortest-path problem. A node records the already covered Query and K/V prefixes; each outgoing edge instantiates one compiled graph, advances coverage, and carries its measured cost. Decode

edges additionally distinguish the graph that initializes a local state from graphs that merge later states, and an edge may cross the visible boundary when its excess cells are masked. Fused graphs, exact compositions, and padded boundary graphs are therefore comparable edges in the same directed acyclic graph. The coverage rules construct legal paths, and target measurements choose the fastest one without assuming that the largest block or fewest launches must win.

**Prefill planning.** Prefill adds a Query dimension and causal dependency. EdgeInfer enumerates supported exact and lightly padded Query heights, then advances through the prompt in causal order. For each Query block, a Decode-style subproblem selects the K/V chain covering its visible prefix; the outer dynamic program selects the Query partition and accumulates graph, memory, and merge costs. Because a Query block depends only on the covered Query prefix and its absolute position, candidates form a directed acyclic graph and admit exact shortest-path search. This jointly chooses Query height, K/V width, boundary padding, and the points at which local states are merged.

**Offline planning and online lookup.** Measurement and search occur before deployment for every supported phase and context state. The selected plans are serialized as a compact table keyed by phase, Query rows, and visible K/V length. Online inference performs one lookup and submits the recorded graph sequence; no search, compilation, or dynamic-shape specialization lies on the token path. New NPU generations require remeasuring the finite shape library, not redesigning the planning algorithm.

### 3.4 Execution: Exact Attention State Composition

**Local state and merge.** For  $Q_i \in \mathbb{R}^{R \times d}$  and K/V block  $(K_j, V_j)$ , graph  $G(R, C)$  computes masked scores  $\hat{Z}_{ij}$  and an unnormalized local state  $(m_{ij}, \ell_{ij}, N_{ij})$ :

$$\begin{aligned} \hat{Z}_{ij} &= Q_i K_j^\top / \sqrt{d} + M_{ij}, \\ m_{ij} &= \max_k \hat{Z}_{ij,k}, \\ (\ell_{ij}, N_{ij}) &= \sum_k e^{\hat{Z}_{ij,k} - m_{ij}} (1, V_{j,k}). \end{aligned} \quad (2)$$

For two states  $A = (m_A, \ell_A, N_A)$  and  $B = (m_B, \ell_B, N_B)$ , let  $m = \max(m_A, m_B)$  and  $\alpha_s = e^{m_s - m}$  for  $s \in \{A, B\}$ . Their exact merge is

$$\begin{aligned} (\ell, N) &= \alpha_A(\ell_A, N_A) + \alpha_B(\ell_B, N_B), \\ O &= N/\ell. \end{aligned} \quad (3)$$

The maximum and denominator states satisfy  $m_{ij}, \ell_{ij} \in \mathbb{R}^{R \times 1}$ , while  $N_{ij} \in \mathbb{R}^{R \times d_v}$ . Retaining the unnormalized numerator  $N_{ij}$  preserves each block’s probability mass. A fully masked block is omitted, equivalently contributing  $(-\infty, 0, 0)$ .

**Why the merge is exact.** For any score  $z$  in block  $j$  and global maximum  $m$ , the identity  $e^{z-m} = e^{m_j-m} e^{z-m_j}$  rescales that block’s local denominator and numerator into the shared global frame. Summing the rescaled terms over all blocks therefore recovers exactly the denominator and

weighted numerator of one softmax over their union. The operation is associative, so graph completion order and reduction topology do not affect the result. EdgeInfer never concatenates independently normalized block outputs.

**Compact execution contract.** Each graph returns  $O(R(d_v + 2))$  state rather than an  $O(RC)$  score matrix. This compact contract limits state traffic, allows tree reduction, and decouples numerical correctness from block placement. Our evaluated implementation keeps all selected graphs on the Hexagon NPU. The same contract can later label independent graphs with NPU, GPU, or CPU backends and merge their states unchanged, making heterogeneous placement an extension of the planner rather than a different Attention algorithm.

## 4 Experiments

### 4.1 Experimental Setup

Our evaluation addresses three questions: (1) Does EdgeInfer reduce long-context Decode and Prefill time? (2) Do real-device costs provide useful guidance for static graph composition? (3) How much does each design choice contribute?

We evaluate a Xiaomi 13 with the Snapdragon 8 Gen 2 mobile platform (SM8550, HTP V73, Android 13) and a OnePlus 12 with the Snapdragon 8 Gen 3 mobile platform (SM8650, HTP V75, Android 16). Both devices execute on the Qualcomm Hexagon NPU through the QNN HTP backend. We export and deploy graphs with Qualcomm AI Runtime 2.43.0 and invoke them through Qualcomm AI Engine Direct 2.18.0. The maximum context is 40,960 tokens. Throughout the paper, Xiaomi 13 SM8550 and OnePlus 12 SM8650 identify the phones, while HTP V73 and HTP V75 identify the HTP generations rather than the SoCs.

The PyTorch ExecuTorch *fixed-maximum-context baseline* (*Fixed*) executes the same  $C_{\max} = 40960$  graph at every context, with neither smaller-graph substitution nor online recompilation. EdgeInfer composes graphs for the current Query length and visible K/V context. Main context, model-depth, and Attention-shape results use W16A16 IEEE FP16. Each ablation compares designs at matched precision and execution boundaries.

### 4.2 Overall Effectiveness and Model-Depth Scaling

Table 1 reports one-decoder-block results from 1K to 40K. Decode includes pre-projection, Attention, and post-projection. Prefill reports the cumulative time of the same decoder block over the complete prompt using 128-row Query blocks. As context grows, Fixed continues to execute the 40K graph, while EdgeInfer follows the useful Attention region.

Figure 3 shows the complete trends. At 40K, EdgeInfer accelerates Decode and Prefill by  $76.9\times$  and  $39.2\times$  on Xiaomi 13 SM8550, and by  $93.4\times$  and  $46.5\times$  on OnePlus 12 SM8650. The gains are larger at shorter contexts because a greater fraction of the fixed 40K graph is redundant.

| Context                  | Decode (ms) |        |              | Prefill (s) |        |              |
|--------------------------|-------------|--------|--------------|-------------|--------|--------------|
|                          | Fixed       | Edge   | Gain         | Fixed       | Edge   | Gain         |
| <i>Xiaomi 13 SM8550</i>  |             |        |              |             |        |              |
| 1K                       | 2343.635    | 1.529  | $1533\times$ | 19.789      | 0.040  | $501\times$  |
| 2K                       | 2345.491    | 2.284  | $1027\times$ | 39.578      | 0.101  | $391\times$  |
| 4K                       | 2346.024    | 3.776  | $621\times$  | 79.155      | 0.295  | $268\times$  |
| 8K                       | 2345.211    | 6.888  | $340\times$  | 158.311     | 0.966  | $164\times$  |
| 16K                      | 2345.001    | 12.757 | $184\times$  | 316.622     | 3.446  | $92\times$   |
| 32K                      | 2346.254    | 24.773 | $95\times$   | 633.244     | 12.957 | $49\times$   |
| 40K                      | 2363.900    | 30.733 | $76.9\times$ | 785.422     | 20.060 | $39.2\times$ |
| <i>OnePlus 12 SM8650</i> |             |        |              |             |        |              |
| 1K                       | 1904.274    | 1.183  | $1610\times$ | 17.173      | 0.032  | $542\times$  |
| 2K                       | 1903.188    | 1.657  | $1149\times$ | 34.346      | 0.080  | $429\times$  |
| 4K                       | 1913.638    | 2.683  | $713\times$  | 68.692      | 0.229  | $300\times$  |
| 8K                       | 1909.969    | 4.726  | $404\times$  | 137.383     | 0.734  | $187\times$  |
| 16K                      | 1904.946    | 8.866  | $215\times$  | 274.766     | 2.575  | $107\times$  |
| 32K                      | 1905.922    | 17.018 | $112\times$  | 549.533     | 9.601  | $57\times$   |
| 40K                      | 1994.460    | 21.343 | $93.4\times$ | 686.916     | 14.779 | $46.5\times$ |

Table 1: Single-block results. Fixed: ExecuTorch maximum-context baseline; Edge: EdgeInfer; Gain: Fixed/Edge.

| Blocks | Xiaomi 13 SM8550 |         |              | OnePlus 12 SM8650 |         |              |
|--------|------------------|---------|--------------|-------------------|---------|--------------|
|        | Fixed            | Edge    | Gain         | Fixed             | Edge    | Gain         |
| 1      | 2363.900         | 30.733  | $76.9\times$ | 1994.460          | 21.343  | $93.4\times$ |
| 2      | 4686.634         | 61.904  | $76\times$   | 3955.182          | 43.047  | $92\times$   |
| 4      | 9396.413         | 130.236 | $72\times$   | 8412.179          | 86.478  | $97\times$   |
| 8      | 18849.308        | 277.169 | $68\times$   | 16702.880         | 174.017 | $96\times$   |
| 16     | —                | —       | —            | 32874.404         | 381.695 | $86\times$   |
| 28     | —                | —       | —            | 55938.707         | 722.513 | $77\times$   |
| 32     | —                | —       | —            | 64534.467         | 829.026 | $78\times$   |

Table 2: 40K multi-block Decode (ms); columns follow Table 1.

**Scaling Across Transformer Decoder Blocks** Table 2 and Figure 4 evaluate stacked decoder-block execution at 40K Decode, with an independent K/V cache for every block instance. Both methods scale steadily with stack depth. EdgeInfer maintains  $68\text{--}76.9\times$  speedups for 1–8 blocks on Xiaomi 13 SM8550 and  $77\text{--}97\times$  for 1–32 blocks on OnePlus 12 SM8650. At 32 blocks on OnePlus 12 SM8650, it reduces execution time from 64.5 s to 0.83 s.

### 4.3 Perception: Hardware Cost Surface

This section reports Attention time for one Transformer decoder block and examines the hardware behavior that motivates our planner.

Figure 5 measures 110 static Attention block shapes on both devices. Latency is not monotonic in  $R \times C$  or FLOPs, and the fastest regions differ between the two NPU generations. Graph shape and device behavior therefore matter jointly; a fixed partition rule cannot reliably identify the best plan.

The measured surface also contains sharp transitions as K/V width grows: reducing graph launches can move execution into a slower hardware region. EdgeInfer therefore

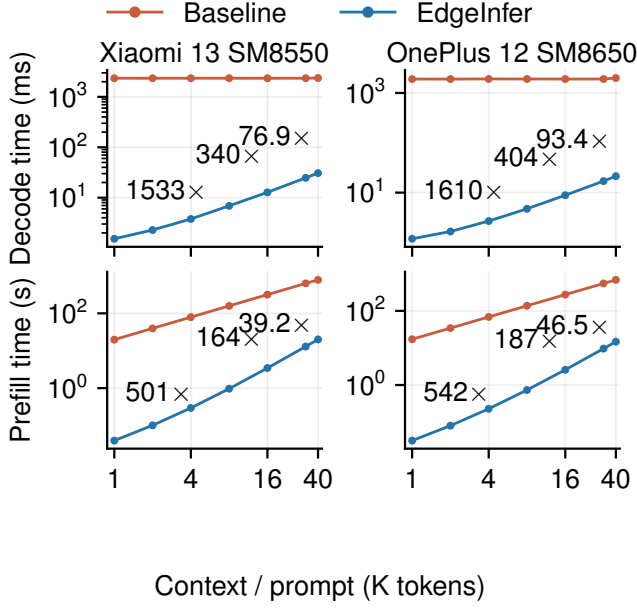


Figure 3: Single-decoder-block performance across context lengths. Decode time is shown above and cumulative Prefill time below; the two columns correspond to Xiaomi 13 SM8550 and OnePlus 12 SM8650. Fixed always uses  $C = 40960$ .

retains several Query and K/V block sizes and lets the measured cost surface choose among them. The device-specific selections in Figure 6(b) demonstrate how this perception stage changes the final plan. Owing to space limits, complete shape data, additional sweeps, cost-model calibration, and exact state-composition checks appear in the supplementary material.

#### 4.4 Decision and Execution Ablations

We next isolate the execution granularity, causal-block pruning, adaptive K/V block width, hardware-aware strategy selection, and adaptive Query-row padding. Each ablation reports execution time and annotates the improvement of the complete EdgeInfer design.

**Static Graph Granularity** Figure 6(d) disables adaptive composition and forces one fixed Attention-block plan. Complete EdgeInfer improves over this restriction by  $3.01\times$  and  $3.46\times$  on the two devices, showing that block decomposition alone is insufficient without context- and device-aware composition.

Table 3 complements this plan-level ablation with a component breakdown against one full 40K Attention graph. Pre- and post-projection remain nearly unchanged, whereas EdgeInfer reduces Attention from 2331.830/1916.650 ms to 29.999/20.435 ms, respectively, on the two devices. The improvement therefore comes from causal execution geometry and graph composition rather than surrounding layer operations.

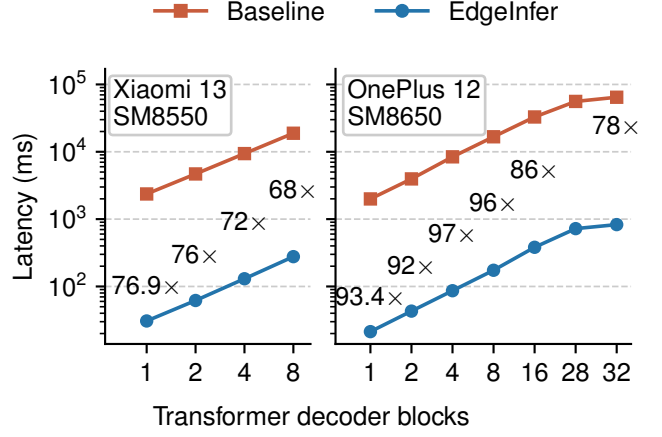


Figure 4: 40K Decode scaling across Transformer decoder blocks. Each block maintains an independent K/V cache; speedups are annotated on the EdgeInfer curves.

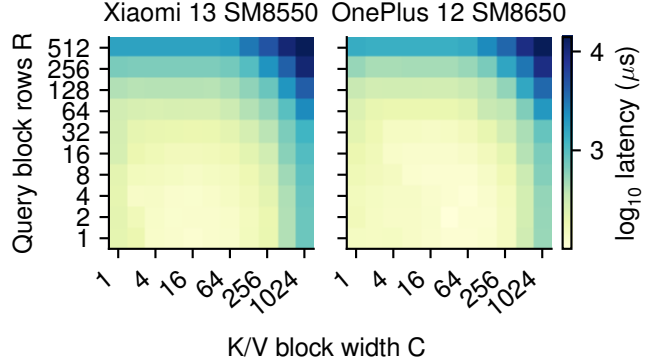


Figure 5: Measured latency of static Attention block shapes.  $R$  is the Query block height and  $C$  is the K/V block width; the two panels correspond to Xiaomi 13 SM8550 and OnePlus 12 SM8650.

**Pruning Causally Invalid Blocks** EdgeInfer multiplies each Query block only by K/V blocks intersecting its visible causal prefix. The ablated variant executes every K/V block for every Query block, including complete upper-right causal blocks. Figure 6(e) shows that pruning increasingly reduces Attention time as context grows, reaching  $1.98\times$  and  $1.96\times$  at 40K.

**Adaptive K/V Block Width** The planner considers a power-of-two family of K/V widths starting at one,  $C \in \{2^i \mid i = 0, \dots, m\}$ . The upper endpoint is determined by the available candidate graph shapes and is not fixed to 1024. For each target length and device, the planner evaluates all admissible width and composition combinations and selects the fastest measured plan. Figure 6(a) compares the selected 40K plan with fixed  $C = 1024$ . Although the largest block minimizes graph launches, it is not fastest on either device. Hardware-aware width selection improves performance by  $1.31\times$  and  $1.62\times$ , confirming that launch count and padding alone are insufficient.



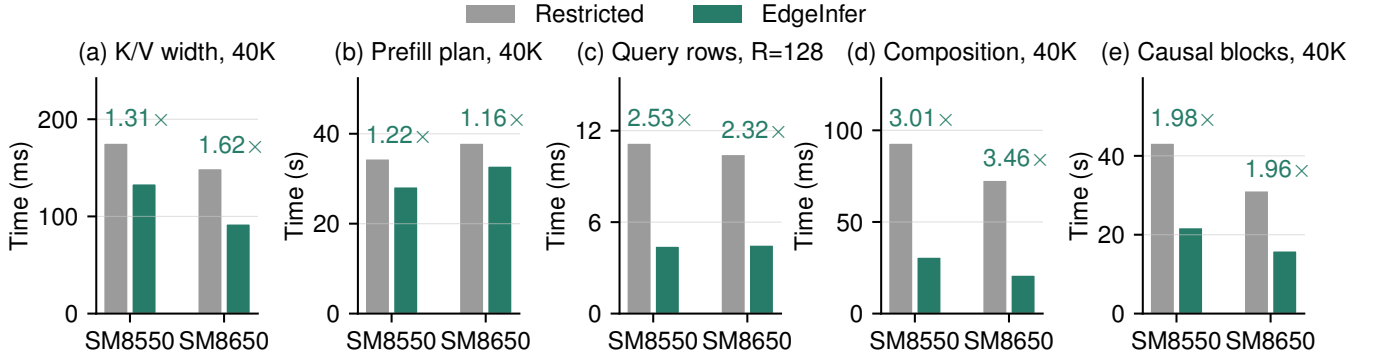


Figure 6: Measured EdgeInfer ablations. (a) Adaptive versus fixed K/V width ( $C = 1024$ ). (b) Hardware-aware versus fixed Prefill planning. (c) Adaptive versus exact Query rows. (d) Adaptive versus fixed block composition. (e) Causal pruning versus all K/V blocks. All use 40K except (c), which fixes  $R = 128$ . Bars show time; labels show speedup over the restricted design.

| Device            | Granularity      | Pre   | Attn.    | Post  |
|-------------------|------------------|-------|----------|-------|
| Xiaomi 13 SM8550  | Full Attention   | 0.349 | 2331.830 | 0.645 |
|                   | EdgeInfer blocks | 0.257 | 29.999   | 0.534 |
| OnePlus 12 SM8650 | Full Attention   | 0.432 | 1916.650 | 0.747 |
|                   | EdgeInfer blocks | 0.208 | 20.435   | 0.420 |

Table 3: Component time in milliseconds for different 40K Decode graph granularities.

**Hardware-Aware Strategy Selection** Figure 6(b) compares EdgeInfer with fixed AR128 and largest-first strategies at 40K Prefill. On Xiaomi 13 SM8550, EdgeInfer selects the largest-first route and improves over fixed AR128 by  $1.22\times$ . On OnePlus 12 SM8650, it selects a different measured-cost composition and improves over largest-first by  $1.16\times$ . The device-specific choices show why no fixed strategy is uniformly optimal.

**Adaptive Query-Row Padding** EdgeInfer selects between exact-height and lightly padded Query blocks, while the restricted design permits exact heights only. Figure 6(c) shows that modest padding can improve hardware utilization despite extra operations, yielding  $2.53\times$  and  $2.32\times$  speedups.

#### 4.5 Implications for Static-Graph Edge Inference

**Static execution need not imply a fixed workload.** Mobile NPUs derive efficiency from ahead-of-time optimization, but this property is often treated as fundamentally incompatible with evolving LLM state. The results reveal a different systems boundary. EdgeInfer leaves every hardware graph immutable and moves adaptation into the choice and composition of graphs. At 40K Decode, this software layer reduces one-decoder-block time from 2363.9/1994.5 ms to 30.733/21.343 ms on the two phones; it does so while preserving the same static-graph execution model. A finite portfolio of static blocks can therefore act as a basis for a much larger dynamic workload space. This changes the deployment question from whether an edge NPU supports dynamic shapes to whether its runtime exposes composable static units

and sufficiently low-overhead graph submission.

**Hardware perception provides a portability boundary.** The heatmaps and strategy ablations show that an efficient block on one NPU generation is not automatically efficient on another. EdgeInfer isolates this device dependence in the measured cost table: causal coverage, state composition, and shortest-path planning remain unchanged, while a new target supplies new measurements for the same finite shape library. This separation is useful for production runtimes because compiler and microarchitectural changes do not require a new Attention algorithm or hand-authored policy. They require a refreshed hardware view from which the same planner derives device-specific decisions.

**The abstraction spans LLM depth and future placement.** Each Transformer decoder block retains its own parameters and K/V cache, yet the block-shape portfolio and planning topology are reusable. The near-linear timing trends in Table 2 and stable gains through 32 blocks show that the single-block execution abstraction persists as the decoder stack grows. The same property also exposes independent K/V-block tasks. The evaluated plans keep the NPU as the primary engine; because each task returns the compact state  $(m, \ell, N)$ , a planner can later attach a backend label to selected tasks and merge them without changing global Attention semantics. Thus EdgeInfer provides a path from NPU-only static execution to NPU-centric heterogeneous execution through one planning interface rather than separate Attention implementations.

## 5 Conclusion

In this paper, we presented EdgeInfer, which adapts static edge-NPU graphs to dynamic Attention through measurement-guided planning and exact state composition. On two Qualcomm mobile platforms, it substantially accelerates long-context Decode and Prefill and retains strong gains through 32 decoder blocks. The results show that immutable graphs can efficiently serve evolving LLM workloads when selected and composed at runtime for each inference state. By separating offline graph compilation and hardware measure-

ment from online state-aware composition, EdgeInfer turns a finite graph portfolio into a portable NPU runtime.

## References

- Agrawal, A.; Kedia, N.; Panwar, A.; Mohan, J.; Kwatra, N.; Gulavani, B. S.; Tumanov, A.; and Ramjee, R. 2024. Taming Throughput-Latency Tradeoff in LLM Inference with Sarathi-Serve. In *USENIX Symposium on Operating Systems Design and Implementation*.
- Alizadeh, K.; Mirzadeh, S. I.; Belenko, D.; Khatamifard, S.; Cho, M.; Del Mundo, C. C.; Rastegari, M.; and Farajtabar, M. 2024. LLM in a Flash: Efficient Large Language Model Inference with Limited Memory. In *Annual Meeting of the Association for Computational Linguistics*.
- Cai, T.; Li, Y.; Geng, Z.; Peng, H.; Lee, J. D.; Chen, D.; and Dao, T. 2024. Medusa: Simple LLM Inference Acceleration Framework with Multiple Decoding Heads. In *International Conference on Machine Learning*, 5209–5235.
- Chen, L.; Feng, D.; Feng, E.; Wang, Y.; Zhao, R.; Xia, Y.; Xu, P.; and Chen, H. 2025. Characterizing Mobile SoC for Accelerating Heterogeneous LLM Inference. In *ACM Symposium on Operating Systems Principles*.
- Chen, T.; Moreau, T.; Jiang, Z.; Zheng, L.; Yan, E.; Shen, H.; Cowan, M.; Wang, L.; Hu, Y.; Ceze, L.; Guestrin, C.; and Krishnamurthy, A. 2018. TVM: An Automated End-to-End Optimizing Compiler for Deep Learning. In *USENIX Symposium on Operating Systems Design and Implementation*.
- Dao, T. 2024. FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning. In *International Conference on Learning Representations*.
- Dao, T.; Fu, D. Y.; Ermon, S.; Rudra, A.; and Ré, C. 2022. FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness. In *Advances in Neural Information Processing Systems*.
- Dettmers, T.; Lewis, M.; Belkada, Y.; and Zettlemoyer, L. 2022. LLM.int8(): 8-Bit Matrix Multiplication for Transformers at Scale. In *Advances in Neural Information Processing Systems*, 30318–30332.
- Ding, Y.; Yu, C. H.; Zheng, B.; Liu, Y.; Wang, Y.; and Pekhimenko, G. 2023. Hidet: Task-Mapping Programming Paradigm for Deep Learning Tensor Programs. In *ACM International Conference on Architectural Support for Programming Languages and Operating Systems*, 370–384.
- Feng, S.; Hou, B.; Jin, H.; Lin, W.; Shao, J.; Lai, R.; Ye, Z.; Zheng, L.; Yu, C. H.; Yu, Y.; and Chen, T. 2023. TensorIR: An Abstraction for Automatic Tensorized Program Optimization. In *ACM International Conference on Architectural Support for Programming Languages and Operating Systems*, 804–817.
- Frantar, E.; Ashkboos, S.; Hoeffler, T.; and Alistarh, D. 2023. GPTQ: Accurate Post-Training Quantization for Generative Pre-trained Transformers. In *International Conference on Learning Representations*.
- Hao, Z.; Wei, J.; Wang, T.; Huang, M.; Jiang, H.; Jiang, S.; Cao, T.; and Ren, J. 2026. Scaling LLM Test-Time Compute with Mobile NPU on Smartphones. In *European Conference on Computer Systems*.
- Jia, Z.; Padon, O.; Thomas, J.; Warszawski, T.; Zaharia, M.; and Aiken, A. 2019. TASO: Optimizing Deep Learning Computation with Automatic Generation of Graph Substitutions. In *ACM Symposium on Operating Systems Principles*, 47–62.
- Kwon, W.; Li, Z.; Zhuang, S.; Sheng, Y.; Zheng, L.; Yu, C. H.; Gonzalez, J. E.; Zhang, H.; and Stoica, I. 2023. Efficient Memory Management for Large Language Model Serving with PagedAttention. In *ACM Symposium on Operating Systems Principles*.
- Leviathan, Y.; Kalman, M.; and Matias, Y. 2023. Fast Inference from Transformers via Speculative Decoding. In *International Conference on Machine Learning*.
- Lin, J.; Tang, J.; Tang, H.; Yang, S.; Chen, W.-M.; Wang, W.-C.; Xiao, G.; Dang, X.; Gan, C.; and Han, S. 2024. AWQ: Activation-aware Weight Quantization for On-Device LLM Compression and Acceleration. In *Conference on Machine Learning and Systems*.
- Lin, Y.; Tang, H.; Yang, S.; Zhang, Z.; Xiao, G.; Gan, C.; and Han, S. 2025. QServe: W4A8KV4 Quantization and System Co-design for Efficient LLM Serving. In *Conference on Machine Learning and Systems*.
- Liu, H.; and Abbeel, P. 2023. Blockwise Parallel Transformer for Large Context Models. In *Advances in Neural Information Processing Systems*.
- Liu, H.; Zaharia, M.; and Abbeel, P. 2024. Ring Attention with Blockwise Transformers for Near-Infinite Context. In *International Conference on Learning Representations*.
- Liu, Z.; Yuan, J.; Jin, H.; Zhong, S.; Xu, Z.; Braverman, V.; Chen, B.; and Hu, X. 2024a. KIVI: A Tuning-Free Asymmetric 2bit Quantization for KV Cache. In *International Conference on Machine Learning*, 32332–32344.
- Liu, Z.; Zhao, C.; Iandola, F.; Lai, C.; Tian, Y.; Fedorov, I.; Xiong, Y.; Chang, E.; Shi, Y.; Krishnamoorthi, R.; Lai, L.; and Chandra, V. 2024b. MobileLLM: Optimizing Sub-Billion Parameter Language Models for On-Device Use Cases. In *International Conference on Machine Learning*, 32431–32454.
- Patel, P.; Choukse, E.; Zhang, C.; Shah, A.; Goiri, Í.; Maleki, S.; and Bianchini, R. 2024. Splitwise: Efficient Generative LLM Inference Using Phase Splitting. In *ACM/IEEE International Symposium on Computer Architecture*.
- Shen, H.; Roesch, J.; Chen, Z.; Chen, W.; Wu, Y.; Li, M.; Sharma, V.; Tatlock, Z.; and Wang, Y. 2021. Nimble: Efficiently Compiling Dynamic Neural Networks for Model Inference. In *Conference on Machine Learning and Systems*.
- Sheng, Y.; Zheng, L.; Yuan, B.; Li, Z.; Ryabinin, M.; Chen, B.; Liang, P.; Ré, C.; Stoica, I.; and Zhang, C. 2023. FlexGen: High-Throughput Generative Inference of Large Language Models with a Single GPU. In *International Conference on Machine Learning*, 31094–31116.
- Tan, F.; Lee, R.; Dudziak, L.; Hu, S. X.; Bhattacharya, S.; Hospedales, T. M.; Tzimiropoulos, G.; and Martínez, B. 2024. MobileQuant: Mobile-friendly Quantization for On-device Language Models. In *Findings of the Association for Computational Linguistics: EMNLP*, 9761–9771.



- Vaswani, A.; Shazeer, N.; Parmar, N.; Uszkoreit, J.; Jones, L.; Gomez, A. N.; Kaiser, L.; and Polosukhin, I. 2017. Attention Is All You Need. In *Advances in Neural Information Processing Systems*, 5998–6008.
- Wei, J.; Cao, S.; Cao, T.; Ma, L.; Wang, L.; Zhang, Y.; and Yang, M. 2025. T-MAC: CPU Renaissance via Table Lookup for Low-Bit LLM Deployment on Edge. In *European Conference on Computer Systems*, 278–292.
- Xiao, G.; Lin, J.; Seznec, M.; Wu, H.; Demouth, J.; and Han, S. 2023. SmoothQuant: Accurate and Efficient Post-Training Quantization for Large Language Models. In *International Conference on Machine Learning*.
- Xiao, G.; Tian, Y.; Chen, B.; Han, S.; and Lewis, M. 2024. Efficient Streaming Language Models with Attention Sinks. In *International Conference on Learning Representations*.
- Xu, D.; Yin, W.; Zhang, H.; Jin, X.; Zhang, Y.; Wei, S.; Xu, M.; and Liu, X. 2025a. EdgeLLM: Fast On-Device LLM Inference with Speculative Decoding. *IEEE Transactions on Mobile Computing*, 24(4): 3256–3271.
- Xu, D.; Zhang, H.; Yang, L.; Liu, R.; Huang, G.; Xu, M.; and Liu, X. 2025b. Fast On-device LLM Inference with NPUs. In *ACM International Conference on Architectural Support for Programming Languages and Operating Systems*.
- Xu, D.; Zhang, H.; Yang, L.; Liu, R.; Xu, M.; and Liu, X. 2024. WiP: Efficient LLM Prefilling with Mobile NPU. In *Workshop on Edge and Mobile Foundation Models*.
- Ye, Z.; Chen, L.; Lai, R.; Lin, W.; Zhang, Y.; Wang, S.; Chen, T.; Kasikci, B.; Grover, V.; Krishnamurthy, A.; and Ceze, L. 2025. FlashInfer: Efficient and Customizable Attention Engine for LLM Inference Serving. In *Conference on Machine Learning and Systems*.
- Yin, W.; Xu, D.; Xu, M.; Huang, G.; and Liu, X. 2026. ShadowNPU: System and Algorithm Co-design for NPU-Centric On-Device LLM Inference. In *ACM International Conference on Mobile Systems, Applications, and Services*.
- Yin, W.; Yi, R.; Xu, D.; Huang, G.; Xu, M.; and Liu, X. 2025. Elastic On-Device LLM Service. In *ACM International Conference on Mobile Computing and Networking*.
- Yu, G.-I.; Jeong, J. S.; Kim, G.-W.; Kim, S.; and Chun, B.-G. 2022. Orca: A Distributed Serving System for Transformer-Based Generative Models. In *USENIX Symposium on Operating Systems Design and Implementation*, 521–538.
- Zhang, Z.; Sheng, Y.; Zhou, T.; Chen, T.; Zheng, L.; Cai, R.; Song, Z.; Tian, Y.; Ré, C.; Barrett, C. W.; Wang, Z.; and Chen, B. 2023. H2O: Heavy-Hitter Oracle for Efficient Generative Inference of Large Language Models. In *Advances in Neural Information Processing Systems*, 34661–34710.
- Zheng, L.; Jia, C.; Sun, M.; Wu, Z.; Yu, C. H.; Haj-Ali, A.; Wang, Y.; Yang, J.; Zhuo, D.; Sen, K.; Gonzalez, J. E.; and Stoica, I. 2020. Ansor: Generating High-Performance Tensor Programs for Deep Learning. In *USENIX Symposium on Operating Systems Design and Implementation*.
- Zhong, Y.; Liu, S.; Chen, J.; Hu, J.; Zhu, Y.; Liu, X.; Jin, X.; and Zhang, H. 2024. DistServe: Disaggregating Prefill and Decoding for Goodput-Optimized Large Language Model Serving. In *USENIX Symposium on Operating Systems Design and Implementation*.
- Zhu, H.; Wu, R.; Diao, Y.; Ke, S.; Li, H.; Zhang, C.; Xue, J.; Ma, L.; Xia, Y.; Cui, W.; Yang, F.; Yang, M.; Zhou, L.; Cidon, A.; and Pekhimenko, G. 2022. ROLLER: Fast and Efficient Tensor Compilation for Deep Learning. In *USENIX Symposium on Operating Systems Design and Implementation*, 233–248.